there is a function, which converts WSG84-GPS coordinates to a mercator projection. How to map mercator projection coordinates to a x & y position in an application window?

/**

- @brief Converts WSG84-GPS coordinates to mercator projection coordinates, used for displaying
- @note Clamping latitude is important because the Mercator projection tends to infinity at the
- poles, and practical Web Mercator usage limits latitude to about 85.0511 degrees.
- @param scale
- @param longitude_deg in decimal degrees (will be converted to radians inside)
- @param latitude_deg in decimal degrees (will be converted to radians inside)
- @return
  */
  coordinates_t coordinates_mercator_project (double scale, double longitude_deg, double latitude_deg)
  {
  const double DEG_TO_RAD = M_PI / 180.0;
  const double MAX_LAT = 85.05112878; // practical Mercator limit

coordinates_t p;

if (latitude_deg > MAX_LAT)
latitude_deg = MAX_LAT;
if (latitude_deg < -MAX_LAT)
latitude_deg = -MAX_LAT;

double lon = longitude_deg * DEG_TO_RAD;
double lat = latitude_deg * DEG_TO_RAD;

p.longitude = (scale * lon);
p.latitude = (scale * log (tan (M_PI / 4.0 + lat / 2.0)));

return p;
}

Yes: after Mercator projection, you still need a second step that converts projected map coordinates into window coordinates using your current view rectangle and window size. In practice, this is just an affine transform: normalize projected coordinates within the visible Mercator bounds, then scale to pixel width and height, with Y usually inverted because screen coordinates grow downward.[1] [2]

Core idea

Your function returns Mercator-space coordinates, not screen-space coordinates. A screen position needs three more things: the Mercator coordinate of the current map center or visible bounds, the zoom/scale of the current view, and the window width and height in pixels.[2] [1]

If your window uses the usual top-left origin, then X increases to the right and Y increases downward, while Mercator Y increases upward, so the Y axis must be flipped during mapping. Libraries that project to screen pixels explicitly mention returning top-left pixel coordinates for canvas/SVG rendering.[2]

Simple formula

Assume you know the visible Mercator rectangle:

- `view_min_x`, `view_max_x`
- `view_min_y`, `view_max_y`

and the window size:

- `window_width`
- `window_height`

Then map a projected point `(mx, my)` to screen pixels like this:[1] [2]

```
double sx = (mx - view_min_x) / (view_max_x - view_min_x) * window_width;
double sy = (view_max_y - my) / (view_max_y - view_min_y) * window_height;
```

The X formula shifts the left edge to 0 and scales to the window width, while the Y formula uses `view_max_y - my` so north appears at the top of the screen. That matches the usual top-left window coordinate convention used by mapping/view libraries.[1] [2]

Center-based formula

Often it is easier to store the map center in Mercator coordinates instead of min/max bounds. If `(cx, cy)` is the Mercator center of the current view, then a point `(mx, my)` becomes:[2]

```
double sx = window_width  * 0.5 + (mx - cx) * pixels_per_mercator_unit;
double sy = window_height * 0.5 - (my - cy) * pixels_per_mercator_unit;
```

Here, `pixels_per_mercator_unit` is your zoom factor for drawing. The minus sign on Y is again the screen-axis flip.[2]

How to get the visible bounds

If your zoom is represented by "Mercator units per screen pixel" or the inverse, you can derive the visible bounds from the center. For example:[2]

```
double half_w = window_width  * 0.5 / pixels_per_mercator_unit;
double half_h = window_height * 0.5 / pixels_per_mercator_unit;

double view_min_x = cx - half_w;
double view_max_x = cx + half_w;
double view_min_y = cy - half_h;
double view_max_y = cy + half_h;
```

Then use the normalization formula from above. This is mathematically equivalent to the center-based form.[2]

With your function

Using your `coordinates_mercator_project`, a complete version could look like this:

```
typedef struct {
    double x;
    double y;
} screen_point_t;

screen_point_t mercator_to_screen(
    coordinates_t merc,
    double center_x,
    double center_y,
    double pixels_per_unit,
    int window_width,
    int window_height)
{
    screen_point_t s;
    s.x = window_width  * 0.5 + (merc.longitude - center_x) * pixels_per_unit;
    s.y = window_height * 0.5 - (merc.latitude  - center_y) * pixels_per_unit;
    return s;
}
```

And usage:

```
coordinates_t p = coordinates_mercator_project(scale, lon_deg, lat_deg);
screen_point_t s = mercator_to_screen(
    p,
    map_center_merc_x,
    map_center_merc_y,
    zoom_pixels_per_unit,
    window_width,
    window_height);
```

This works because the first function converts geographic coordinates into Mercator coordinates, and the second converts Mercator coordinates into top-left-origin window pixels relative to your current camera/view.[1] [2]

Important detail

Be careful not to use one `scale` value for two different meanings unless you intend that. In your projection code, `scale` multiplies the Mercator coordinates themselves, while in rendering you also need a view zoom factor that says how many pixels correspond to one Mercator unit; those can be the same by design, but conceptually they are different roles.[2]

Practical example

If your app window is `800x600`, your map center in Mercator space is `(1000, 500)`, and your point is `(1010, 490)` with `2` pixels per Mercator unit, then:[2]

- `x = 400 + (1010 - 1000) * 2 = 420`[2]
- `y = 300 - (490 - 500) * 2 = 320`[2]

So the point is drawn at `(420, 320)`.[2]

Recommendation

The cleanest design is:

1. Convert lon/lat to Mercator.
2. Store your current camera as Mercator center + zoom.
3. Convert Mercator to screen with translation, scaling, and Y flip.[1] [2]

If you want, I can also give you:

- a full `lat/lon -> screen x/y` function in C,
- support for panning and zooming,
- or the inverse `screen x/y -> lat/lon` function.

⁂

how these "Mercator units" corresponds to a distance (like in meters) in real world?

Mercator “units” are not directly meters; they are a scaled Easting/Northing that you must tie to your chosen map scale (and latitude) to get a meaningful distance in meters.[11] [12]

Concept: a scale factor

At the equator, Web Mercator is usually defined so that:

- the Earth's ellipsoid is approximated by a sphere of radius about $R \approx 6\,378\,137$ m, and
- the projected X unit is essentially "radians of longitude × R", i.e., $x = R\,\lambda$.[13] [11]

Then:

- 1 Mercator unit ≈ 1 meter at the equator, if you set your drawing scale such that "1 Mercator unit = 1 meter" there.[14] [11]
- If you instead use, say, `scale = 2.0` in your projection, then 1 Mercator unit corresponds to 0.5 m in real-world length at the equator (because everything is stretched horizontally by 2×).[15] [14]

So in general:

$$\text{distance in meters} \approx (\text{Mercator units}) \times \text{scale_factor}^{-1}$$

where `scale_factor` is the factor you pass into your projection (or later into your viewport) that links projected Mercator coordinates to meters at the reference latitude.[16] [17]

Latitude dependence

Because Mercator stretches vertically with latitude, the number of meters per Mercator unit depends on where you measure it.[18] [19]

- At higher latitudes, the scale factor increases roughly by $\sec\varphi$ (1 / cos φ), so:
  - 1 horizontal Mercator unit at latitude φ ≈ 1 m × $R/R_{\text{eq}} \times \sec\varphi$
  - in practice, at 60°, 1 Mercator unit corresponds to about 2 m along a meridian, while at the equator it's about 1 m per unit.[12] [18]

If you want precise "distance in meters" from Mercator differences, you must:

1. choose a reference latitude (often the map center),
2. compute the Mercator scale factor at that latitude,
3. multiply Mercator distances (Δx, Δy) by scale_factor^{-1}.[12] [16]

Practical rule-of-thumb

If you set your `scale` so that:

- at your target latitude, "1 Mercator unit" ≈ 1 m in your map,
- then the Euclidean distance between two Mercator points is approximately the distance in meters, assuming you stay near that latitude and the map is not too large.[17] [12]

If you tell me your `scale` and the latitude you care about, I can derive an explicit "1 Mercator unit = X meters" factor for your app.

⁂

ok. go one prompt back and give a full lat/lon -> screen x/y function in C

Sure — below is a complete C function that goes directly from latitude/longitude to screen `x`, `y`, using the same Mercator math as your function and a map center + zoom in Mercator units. The key steps are: clamp latitude to the practical Mercator limit, project lon/lat into Mercator coordinates, subtract the current map center, scale into pixels, and flip Y so north appears upward while screen Y grows downward.[26] [27]

Full function

```
#include <math.h>

typedef struct
{
    double x;
    double y;
} screen_point_t;

typedef struct
{
    double x;   /* mercator x */
    double y;   /* mercator y */
} mercator_point_t;

/* Convert WGS84 lon/lat in degrees to Mercator coordinates */
static mercator_point_t
wgs84_to_mercator(double mercator_scale, double longitude_deg, double latitude_deg)
{
    const double DEG_TO_RAD = M_PI / 180.0;
    const double MAX_LAT = 85.05112878;

    if (latitude_deg > MAX_LAT)  latitude_deg = MAX_LAT;
    if (latitude_deg < -MAX_LAT) latitude_deg = -MAX_LAT;

    double lon = longitude_deg * DEG_TO_RAD;
    double lat = latitude_deg  * DEG_TO_RAD;

    mercator_point_t p;
    p.x = mercator_scale * lon;
    p.y = mercator_scale * log(tan(M_PI / 4.0 + lat / 2.0));
    return p;
}

/*
 * Convert WGS84 lon/lat directly to screen coordinates.
 *
 * Parameters:
 *   mercator_scale         - scale used inside Mercator projection
 *   longitude_deg          - point longitude in degrees
 *   latitude_deg           - point latitude in degrees
 *   center_longitude_deg   - map center longitude in degrees
```

```
 *   center_latitude_deg    - map center latitude in degrees
 *   pixels_per_mercator    - zoom: how many screen pixels per Mercator unit
 *   window_width           - window width in pixels
 *   window_height          - window height in pixels
 *
 * Returns:
 *   screen x,y in pixels, with origin at top-left
 */
screen_point_t
latlon_to_screen_xy(double mercator_scale,
                    double longitude_deg,
                    double latitude_deg,
                    double center_longitude_deg,
                    double center_latitude_deg,
                    double pixels_per_mercator,
                    int window_width,
                    int window_height)
{
    mercator_point_t p = wgs84_to_mercator(mercator_scale, longitude_deg, latitude_
    mercator_point_t c = wgs84_to_mercator(mercator_scale, center_longitude_deg, ce

    screen_point_t s;
    s.x = (window_width  * 0.5) + (p.x - c.x) * pixels_per_mercator;
    s.y = (window_height * 0.5) - (p.y - c.y) * pixels_per_mercator;

    return s;
}
```

This version uses the center of the window as the map center, which is a common camera model for 2D map rendering. The Y expression subtracts Mercator Y from the screen center because projected north is positive upward, but screen coordinates usually increase downward.[27] [26]

Example usage

```
screen_point_t pt = latlon_to_screen_xy(
    1.0,        /* mercator_scale */
    19.0402,    /* point lon  */
    49.8224,    /* point lat  */
    19.0000,    /* map center lon */
    49.8000,    /* map center lat */
    50000.0,    /* pixels per mercator unit */
    1280,       /* window width */
    720         /* window height */
);
```

After that, `pt.x` and `pt.y` are the pixel coordinates where you draw the point in your application window. If the point is outside the current viewport, the returned coordinates will naturally be outside `[0,width]` or `[0,height]`.[28] [26]

If you want pan offsets

If your renderer uses a top-left map origin instead of a centered camera, you can convert through explicit pan offsets:

```
screen_x = (p.x * pixels_per_mercator) + pan_x;
screen_y = (-p.y * pixels_per_mercator) + pan_y;
```

That is the same math, just expressed with a translation vector instead of a center coordinate. The centered-camera version is usually easier to reason about for interactive zooming and panning.[26] [28]

Important note

Your `mercator_scale` and `pixels_per_mercator` are different concepts even though both affect size on screen. `mercator_scale` defines the size of the projected coordinate system itself, while `pixels_per_mercator` is your viewport zoom that converts projected coordinates into screen pixels.[27] [28]

If you want, I can next give you one of these:

- a version that uses a bounding box instead of map center,
- a matching `screen x/y -> lat/lon` inverse function,
- or a version adapted to SDL/OpenGL.

⁂

1. https://visgl.github.io/math.gl/docs/modules/web-mercator/api-reference/web-mercator-utils
2. https://en.wikipedia.org/wiki/Transverse_Mercator_projection
3. https://visgl.github.io/math.gl/docs/modules/web-mercator/api-reference/web-mercator-viewport
4. https://www.mapwindow.org/mapwingis/group__map__coordinates.html
5. https://deck.gl/docs/api-reference/core/web-mercator-viewport
6. https://tangrams.readthedocs.io/en/main/Overviews/Projections-Overview/
7. https://gambaswiki.org/wiki/comp/gb.opengl.glu/glu/project?l=fr&nh
8. https://stackoverflow.com/questions/64937112/map-web-mercator-pixel-coordinates-to-screen-pixel
9. https://stackoverflow.com/questions/14329691/convert-latitude-longitude-point-to-a-pixels-x-y-on-mercator-projection
10. https://pro.arcgis.com/en/pro-app/latest/help/mapping/properties/coordinate-systems-and-projections.htm
11. https://gist.github.com/phamquyhai/92d95154c45ddea6fe1902e8824a22fd
12. https://docs.geodesk.com/python/mercator
13. https://app.unpkg.com/maplibre-gl@5.2.0/files/src/geo/mercator_coordinate.ts
14. https://surferhelp.goldensoftware.com/projections/Mercator_Projection.htm

15. https://surferhelp.goldensoftware.com/projections/Transverse_Mercator_Projection.htm
16. https://cdn.jsdelivr.net/npm/maplibre-gl@5.2.0/src/geo/mercator_coordinate.ts
17. https://api.visicom.ua/en/products/tiles/theoretical-info
18. https://infogalactic.com/info/Mercator_projection
19. https://en.wikipedia.org/wiki/Mercator_projection
20. https://www.calculatorsconversion.com/en/calculator-coordinates-converter-must-have-best-tool/
21. https://proj.org/en/stable/operations/projections/merc.html
22. https://ccv.eng.wayne.edu/reference/mercator-15dec2015.pdf
23. https://tchayen.com/web-mercator-projection
24. https://hypar-io.github.io/Elements/api/Elements.Spatial.MercatorProjection.html
25. https://stackoverflow.com/questions/26406151/d3-mercator-projection-measuring-distance-or-converting-in-meters
26. https://stackoverflow.com/questions/64937112/map-web-mercator-pixel-coordinates-to-screen-pixel
27. https://en.wikipedia.org/wiki/Web_Mercator_projection
28. https://www.npmjs.com/package/viewport-mercator-project/v/4.0.0-alpha.3
29. https://en.wikipedia.org/wiki/Mercator_projection
30. https://stackoverflow.com/questions/14329691/convert-latitude-longitude-point-to-a-pixels-x-y-on-mercator-projection
31. https://visgl.github.io/math.gl/docs/modules/web-mercator/api-reference/web-mercator-utils
32. https://www.reddit.com/r/learnprogramming/comments/1qeeq2/c_converting_latlong_to_pixels_on_a_mercator/
33. https://api.kde.org/marble/html/MercatorProjection_8h_source.html
34. https://github.com/uber-archive/viewport-mercator-project/blob/master/docs/api-reference/web-mercator-viewport.md
35. https://gist.github.com/aaronhoffman/f53e1852ca289e4a806c062f97a18f05